

Transaction Control Language and Stored Procedure

Transaction Control Language

Definition

- ❖ Transaction Control Language (TCL) is used to manage transactions in a database.
- ❖ It ensures the integrity and consistency of data by controlling the changes made during a transaction.
- ❖ TCL commands are essential to make sure data is saved or discarded appropriately in SQL Server.

Advantage Using TCL

Using **Transaction Control Language (TCL)** in database operations offers several important advantages, particularly in maintaining data integrity, consistency, and reliability.

"TCL helps database processing to implement Atomicity, Consistency, Isolation, Durability (ACID) principles"

Advantage Using TCL (Atomicity)

Ensures Atomicity that a series of database operations (such as multiple inserts, updates, or deletes) are treated as a single unit.

Either **all operations succeed** and are committed to the database, or **none of them** are applied (via *ROLLBACK*), ensuring that the database is always in a consistent state.

Example: If you're transferring money from one account to another, both the debit and credit must happen together. If one fails, the entire transaction should be rolled back to avoid inconsistencies.

Advantage Using TCL (Atomicity)

```
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;    -- Step 1
UPDATE accounts SET balance = balance + 100 WHERE id = 2;    -- Step 2

-- If both succeed:
COMMIT;

-- If any step fails:
ROLLBACK;
```

If step 2 fails (e.g., account 2 doesn't exist), the database *automatically rolls back* step 1

Advantage Using TCL (Consistency)

TCL makes sure your database transitions from one **valid state** to another. If any part of a transaction fails, **ROLLBACK** reverts everything, preventing partial or corrupted data from being saved.

A transaction must take the database from one **valid state** to another **valid state**, obeying all rules such as constraints, triggers, and data types.

Example:

A rule says that an account balance cannot be negative.

If a transaction would make the balance -100, the database rejects it.

Advantage Using TCL (Consistency)

we have a constraint:

```
ALTER TABLE accounts  
ADD CONSTRAINT balance_non_negative CHECK (balance >= 0);
```

If a transaction tries this:

```
BEGIN TRANSACTION;  
  
UPDATE accounts SET balance = balance - 2000 WHERE id = 1; -- Causes -500  
  
COMMIT;
```

The database will **reject** the commit because it violates the constraint.
Thus, the database remains consistent.

Advantage Using TCL (Isolation)

Transactions running at the same time must not interfere with each other.

The final result should be the same as if the transactions were executed **sequentially**.

Example:

❖ Two people try to buy the **last ticket** at the same time.

Without isolation → both might succeed incorrectly.

With isolation → only **one** transaction succeeds; the other must wait or fail.

Advantage Using TCL (Isolation)

using **SERIALIZABLE** isolation level:

Transaction A:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
BEGIN TRANSACTION;  
  
SELECT * FROM tickets WHERE id = 10;  
  
-- Finds: 1 ticket left
```

Transaction B (running at same time)

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
BEGIN TRANSACTION;  
  
SELECT * FROM tickets WHERE id = 10;  
  
-- Also sees: 1 ticket left
```

When both try to update:

```
UPDATE tickets SET quantity = quantity - 1 WHERE id = 10;
```

Only one transaction will succeed.

The other will produce an error like:

```
ERROR: could not serialize access due to concurrent update
```

Advantage Using TCL (Durability)

Once a transaction is **committed**, its result must be permanently saved, even if the system crashes immediately afterward.

Example:

After a successful money transfer and commit, the result stays saved even if:

- ✓ the server loses power,
- ✓ the system crashes,
- ✓ the database is restarted.

Advantage Using TCL (Durability)

```
BEGIN TRANSACTION;  
  
INSERT INTO logs(action, time)  
VALUES ('User A transferred 100 to User B', NOW());  
  
COMMIT;
```

Even if the database server crashes **immediately after COMMIT**, the log entry is permanently saved because the DB writes it to durable storage (WAL / redo log)

Advantage Using TCL (Error Handling & Recovery)

ROLLBACK allows for the reversal of any changes made during a transaction if an error occurs. This ensures that an operation can fail without leaving the database in a corrupted or inconsistent state.

Example: If a database update is interrupted by a failure (e.g., a network crash), using TCL ensures that the changes made during the transaction are undone, restoring the system to its previous state.

Advantage Using TCL (Savepoints)

TCL allows the use of savepoints (using the `SAVEPOINT` command), which let you mark intermediate points within a transaction.

Example:

If a multi-step process (like a large data import) is ongoing, you can create savepoints at key stages. If an error occurs, you can rollback to the last savepoint instead of discarding all progress.

Improves System Integrity Across Multiple Operations

Complex applications often require multiple, interdependent operations to succeed together. TCL helps ensure that all interdependent operations are either fully executed or fully rolled back.

Example: In a reservation system, booking a seat, charging the customer, and sending a confirmation email might all need to happen as a single transaction. If any part fails, the entire transaction should be rolled back to maintain system integrity.

- ❖ **COMMIT** – Makes all changes permanent.
- ❖ **ROLLBACK** – Undoes changes made during the current transaction.
- ❖ **SAVEPOINT** – Creates a point within a transaction to which you can later roll back.
- ❖ **SET TRANSACTION ISOLATION LEVEL** – Sets the isolation level for the transaction.

COMMIT

Usage

“COMMIT;”

BEGIN TRANSACTION;

UPDATE Saving

SET Amount = Amount-1000 WHERE CompanyId = 1 and TypeCustomer=1;

UPDATE Saving

SET Amount = Amount+1000 WHERE CompanyId = 1 and TypeCustomer=2 and CustomerId=200;

COMMIT;

After the COMMIT command, the update to the Employees table is permanent and cannot be rolled back.

ROLLBACK

Usage

“Rollback;”

BEGIN TRANSACTION;

UPDATE Saving

SET Amount = Amount-1000 WHERE CompanyId = 1 and TypeCustomer=1;

COMMIT;

UPDATE Saving

SET Amount = Amount+1000 WHERE CompanyId = 1 and TypeCustomer=2 and CustomerId=200;

Rollback;

Since the ROLLBACK was executed, the update to the Employees table is undone.

Usage

“Save Transaction ;”

BEGIN TRANSACTION;

UPDATE Saving

SET Amount = Amount-1000 WHERE CompanyId = 1 and TypeCustomer=1;

UPDATE Saving

SET Amount = Amount+1000 WHERE CompanyId = 1 and TypeCustomer=2 and CustomerId=200;

SAVE TRANSACTION **SavePoint1**;

UPDATE Saving

SET Amount = Amount-700 WHERE CompanyId = 1 and TypeCustomer=1;

UPDATE Saving

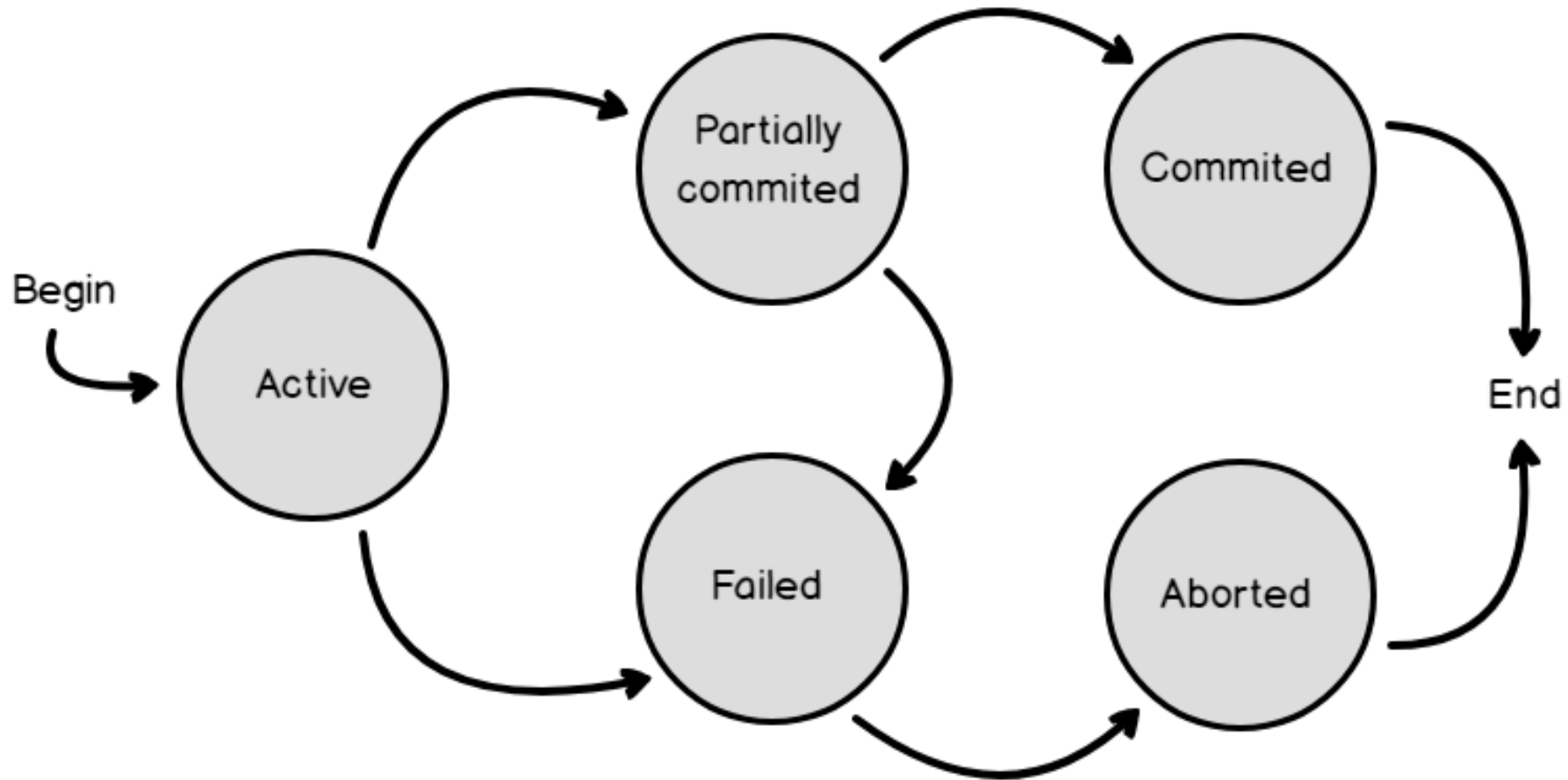
SET Amount = Amount+700 WHERE CompanyId = 1 and TypeCustomer=2 and CustomerId=75;

ROLLBACK TRANSACTION **SavePoint1**;

COMMIT;

Only the third and fourth update is rolled back, and the first and second update is kept.

Try Catch



Try Catch

BEGIN TRY

BEGIN TRANSACTION;

UPDATE Saving

SET Amount = Amount-1000 WHERE CompanyId = 1 and TypeCustomer=1;

UPDATE Saving

SET Amount = Amount+1000 WHERE CompanyId = 1 and TypeCustomer=2 and

CustomerId=200;

COMMIT TRANSACTION;

END TRY

BEGIN CATCH

-- If any error occurs in the main transaction, rollback the entire transaction

ROLLBACK TRANSACTION;

PRINT 'Error in the main transaction: ' + ERROR_MESSAGE();

END CATCH;

Try Catch (2)

BEGIN TRY

BEGIN TRANSACTION;

UPDATE Saving -- (a)

SET Amount = Amount-1000 WHERE CompanyId = 1 and TypeCustomer=1;

UPDATE Saving

SET Amount = Amount+1000 WHERE CompanyId = 1 and TypeCustomer=2 and CustomerId=200;

SAVE TRANSACTION SavePoint1;

BEGIN TRY

BEGIN TRANSACTION;

UPDATE Saving -- (b)

SET Amount = Amount+1000000000 WHERE CompanyId = 1 and TypeCustomer=1;

SAVE TRANSACTION SavePoint2;

END TRY

BEGIN CATCH

-- If any error occurs in the main transaction, rollback the entire transaction

ROLLBACK TRANSACTION SavePoint1; -- (c)

PRINT 'Error in the main transaction: ' + ERROR_MESSAGE();

END CATCH;

--- Any action -- (d)

COMMIT TRANSACTION;

END TRY

BEGIN CATCH -- (e)

-- If any error occurs in the main transaction, rollback the entire transaction

ROLLBACK TRANSACTION; -- (f)

PRINT 'Error in the main transaction: ' + ERROR_MESSAGE();

END CATCH;

Stored Procedure

Stored Procedure

- ❖ A stored procedure is a precompiled collection of one or more SQL statements that can be executed as a single unit.
- ❖ Purpose:
 - Encapsulate logic for reuse.
 - Simplify repetitive tasks.
 - Improve performance by reducing network traffic.

Why Use Stored Procedure

- ❖ Encapsulation: Hide the complexity of SQL queries from end users or applications.
- ❖ Reusability: Write once, use many times.
- ❖ Performance: Stored procedures are precompiled, reducing query execution time.
- ❖ Security: Prevent SQL injection by validating inputs inside the procedure.
- ❖ Maintainability: Centralize logic, making it easier to update and debug..

Component Stored Procedure

- ❖ Name: The stored procedure's unique name.
- ❖ Parameters: [Optionally] Input or output parameters.
- ❖ SQL Logic: The actual SQL statements inside the procedure.
- ❖ Return Type: [Optionally] returns a value (using RETURN)

Format Stored Procedure

```
CREATE PROCEDURE procedure_name  
@param1 datatype, @param2 datatype AS  
BEGIN  
    -- SQL Statements  
END;
```

Sample Stored Procedure

Store Procedure Without Return Value

```
CREATE PROCEDURE GetEmployeeDetails
    @EmployeeID INT
AS
BEGIN
    SELECT Name,
           Position,
           Department
    FROM Employees
    WHERE EmployeeID = @EmployeeID;
END;
```

Store Procedure Without Return Value

```
CREATE PROCEDURE GetCustomerOrderCount
    @CustomerID INT,
    @OrderCount INT OUTPUT -- return value
AS
BEGIN
    SELECT @OrderCount = COUNT(*)
    FROM Orders
    WHERE CustomerID = @CustomerID;
END;
```

Exercise

1. Create Procedure with TCL Insert on your Final Project
2. Create Procedure with TCL Update on your Final Project
3. Create Procedure with TCL Delete on your Final Project
4. Create Procedure implement view on your Final Project



Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,
niscaya ilmu yang sedikit pun akan bermanfaat baginya